

Datenbankentwurf für eine Buchtausch-App

Portfolioteil 2 – Erarbeitungs-/Reflexionsphase

Tizian Senger

26. September 2026

DLBDSPBDM01_D – Data-Mart-Erstellung in SQL

Matrikelnummer: IU14143428

<https://github.com/TizianSenger/IUDatabaseProject>

Agenda

Überblick

Tabellen und SQL-Statements

Dreifachbeziehungen

Constraints und Geschäftsregeln

Zusammenfassung und Reflexion

Überblick

Überblick über das Datenbankdesign

Verwendetes DBMS: SQLite 3 – serverlos, eine einzelne Datei, voll SQL-fähig (CHECK, FOREIGN KEY, TRIGGER, JOIN); laut Aufgabenstellung frei wählbar.

Anzahl Tabellen: 17 (13 Entitäten + 2 Dreifachbeziehungstabellen ausleihe/verfügbarkeit + 2 M:N-Tabellen buch_autor/buch_genre).

Normalform: 3. Normalform (3NF) – keine transitiven Abhängigkeiten, Autor/Verlag/Genre/Sprache konsequent ausgelagert.

Gegenüber Phase 1 umgesetztes Tutor-Feedback:

BUCH (Werk) und BUCHEXEMPLAR (physische Kopie inkl. Eigentümer:in) getrennt.

BEWERTUNG per eindeutigem FK an ausleihe_id gekoppelt statt eigener Dreifachbeziehung.

Durchgängige CHECK-Constraints (Sterne 1–5, Versandkosten ≥ 0 , startzeit < endzeit).

Negativtestfälle für alle zentralen Geschäftsregeln (siehe Folien 20–22).

Tabellen und SQL-Statements

Tabelle: NUTZER

CREATE-Statement:

```
CREATE TABLE nutzer (  
  nutzer_id      INTEGER PRIMARY KEY AUTOINCREMENT,  
  vorname        VARCHAR(50) NOT NULL,  
  nachname       VARCHAR(50) NOT NULL,  
  email          VARCHAR(100) NOT NULL UNIQUE,  
  passwort_hash  VARCHAR(255) NOT NULL,  
  telefon        VARCHAR(20),  
  rolle          VARCHAR(10) NOT NULL  
                DEFAULT 'nutzer'  
                CHECK (rolle IN ('nutzer','admin')),  
  registriert_am DATE NOT NULL  
                DEFAULT CURRENT_DATE  
);
```

Testfall: SELECT nutzer_id, vorname, nachname,
email FROM nutzer WHERE rolle = 'nutzer'; (11
Treffer)

Query Results

SQL Query Editor (click to edit query)

SELECT nutzer_id, vorname, nachname, email FROM nutzer WHERE rolle = 'nutzer';

Execute Query Close

Filter results

nutzer_id	vorname	nachname	email
1	Anna	Bauer	anna.bauer@example.com
2	Ben	Fischer	ben.fischer@example.com
3	Clara	Hoffmann	clara.hoffmann@example.com
4	David	Müller	david.mueller@example.com
5	Emma	Schmidt	emma.schmidt@example.com
6	Felix	Wagner	felix.wagner@example.com
7	Giulia	Chen	giulia.chen@example.com
8	Hannah	Engel	hannah.engel@example.com
9	Ivo	Hausmann	ivo.hausmann@example.com
10	Johanna	Meier	johanna.meier@example.com
11	Karl	Demmeisen	karl.demmeisen@example.com

Tabelle: ADRESSE

CREATE-Statement:

```
CREATE TABLE adresse (  
  adresse_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  nutzer_id INTEGER NOT NULL  
    REFERENCES nutzer(nutzer_id)  
    ON DELETE CASCADE,  
  strasse VARCHAR(100) NOT NULL,  
  hausnummer VARCHAR(10) NOT NULL,  
  plz VARCHAR(10) NOT NULL,  
  stadt VARCHAR(50) NOT NULL,  
  land VARCHAR(50) NOT NULL  
    DEFAULT 'Deutschland'  
);
```

Testfall: Adresse von Anna Bauer per
JOIN adresse-nutzer →
Musterstrasse 12, 10115 Berlin.



Query Results

1. SQL Query Editor (click to collapse)

```
SELECT a.adresse_id, a.hausnummer, a.plz, a.stadt,  
       b.nutzer_id, b.name, b.vorname, b.email, b.password  
FROM adresse a JOIN nutzer b ON a.nutzer_id = b.nutzer_id  
WHERE b.vorname = 'Anna' AND b.nachname = 'Bauer';
```

Execute Query Close

Filter results: 1 row selected

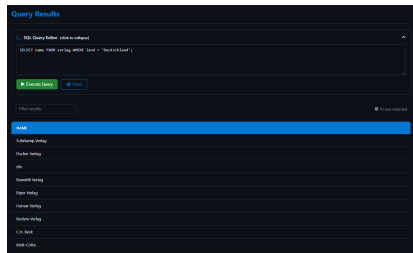
adresse_id	hausnummer	plz	stadt
MUSTERID12	12	10115	Berlin

Tabelle: VERLAG

CREATE-Statement:

```
CREATE TABLE verlag (  
  verlag_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  land VARCHAR(50)  
);
```

Testfall: SELECT name FROM verlag
WHERE land = 'Deutschland'; (9 von
10 Verlagen, Diogenes ist Schweizer
Verlag)



The screenshot shows a web-based database interface. At the top, it says 'Query Results'. Below that, the SQL query is displayed: 'SELECT name FROM verlag WHERE land = 'Deutschland';'. There are buttons for 'Execute Query' and 'Clear'. Below the query, it says 'Filter results:'. The results are shown in a table with a blue header row labeled 'NAME'. The table contains 9 rows of publisher names: 'Suhrkamp Verlag', 'Piper Verlag', 'cbl', 'Rowohlt Verlag', 'Droemer Verlag', 'Harner Verlag', 'Fischer Verlag', 'C.H. Beck', and 'Kohl Verlag'.

NAME
Suhrkamp Verlag
Piper Verlag
cbl
Rowohlt Verlag
Droemer Verlag
Harner Verlag
Fischer Verlag
C.H. Beck
Kohl Verlag

Tabelle: SPRACHE

CREATE-Statement:

```
CREATE TABLE sprache (  
  sprache_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  bezeichnung VARCHAR(50) NOT NULL  
    UNIQUE,  
  iso_code CHAR(2) NOT NULL UNIQUE  
);
```

Testfall: SELECT bezeichnung,
iso_code FROM sprache WHERE
bezeichnung = 'Deutsch'; →
('Deutsch', 'DE')

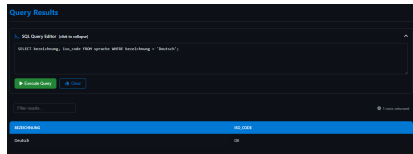


Tabelle: GENRE

CREATE-Statement:

```
CREATE TABLE genre (  
  genre_id    INTEGER PRIMARY KEY  
             AUTOINCREMENT,  
  bezeichnung VARCHAR(50) NOT NULL  
             UNIQUE  
);
```

Testfall: Alle Buecher im Genre *Roman*
per JOIN buch_genre – 5 Treffer (Die
Verwandlung, Der Steppenwolf, Der
Zauberberg, Das Geisterhaus,
Schachnovelle).



The screenshot shows a SQL query editor with the following query:

```
SELECT b.id, t1.title FROM buch b  
JOIN buch_genre bg ON b.id = bg.buch_id  
JOIN genre g ON g.genre_id = bg.genre_id  
WHERE g.bezeichnung = 'Roman'
```

Below the query, there are buttons for 'Execute Query' and 'Close'. The results section shows 5 rows returned, with a table containing the following data:

id
Die Verwandlung
Der Steppenwolf
Der Zauberberg
Das Geisterhaus
Schachnovelle

Tabelle: AUTOR

CREATE-Statement:

```
CREATE TABLE autor (  
  autor_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  vorname VARCHAR(50) NOT NULL,  
  nachname VARCHAR(50) NOT NULL  
);
```

Testfall: Alle Buecher von Franz Kafka per
JOIN buch_autor → *Die Verwandlung*.

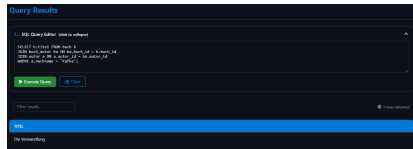


Tabelle: BUCH

CREATE-Statement:

```
CREATE TABLE buch (  
  buch_id    INTEGER PRIMARY KEY  
            AUTOINCREMENT,  
  titel      VARCHAR(150) NOT NULL,  
  verlag_id  INTEGER  
            REFERENCES verlag(verlag_id),  
  sprache_id INTEGER NOT NULL  
            REFERENCES sprache(sprache_id),  
  erscheinungsjahr INTEGER NOT NULL  
            CHECK (erscheinungsjahr  
                  BETWEEN 1450 AND 2100)  
);
```

Testfall: Werk-Stammdaten inkl.
Verlag/Sprache per JOIN – 12
Buecher, z. B. ('Die Verwandlung',
'Fischer Verlag', 'Deutsch', 1915).

Query Results

SELECT buch_id, titel, verlag_id, verlag, sprache_id, sprache, erscheinungsjahr
FROM buch
WHERE verlag_id = 1001 AND sprache_id = 1001 AND erscheinungsjahr
BETWEEN 1450 AND 2100

12 rows

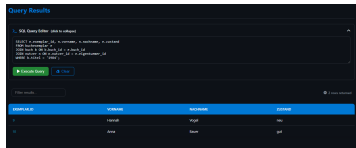
buch_id	titel	verlag_id	verlag	sprache_id	sprache	erscheinungsjahr
1001	Die Verwandlung	1001	Fischer Verlag	1001	Deutsch	1915
1002	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1003	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1004	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1005	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1006	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1007	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1008	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1009	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1010	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1011	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915
1012	Der Schimmelreiter	1001	Fischer Verlag	1001	Deutsch	1915

Tabelle: BUCHEXEMPLAR

CREATE-Statement:

```
CREATE TABLE buchexemplar (  
  exemplar_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  buch_id INTEGER NOT NULL  
    REFERENCES buch(buch_id),  
  eigentuemer_id INTEGER NOT NULL  
    REFERENCES nutzer(nutzer_id)  
    ON DELETE CASCADE,  
  zustand VARCHAR(20) NOT NULL  
    CHECK (zustand IN ('neu', 'gut',  
      'gebraucht', 'stark_gebraucht')),  
  erstellt_am DATE NOT NULL  
    DEFAULT CURRENT_DATE  
);
```

Testfall: Beide Exemplare des Werks 1984 – Eigentüemer Hannah Vogel (neu) und Anna Bauer (gut). Zeigt den Mehrwert der Trennung Buch/Buchexemplar aus dem Tutor-Feedback.



Query Results

SQL Query Editor (data to column)

```
SELECT * FROM buchexemplar  
WHERE buch_id = 1984  
ORDER BY eigentuemer_id
```

Execute Query | Close

Filter results: 2 rows selected

exemplar_id	buch_id	eigentuemer_id	zustand
1	1984	1001	neu
2	1984	1002	gut

Tabelle: ZEITSLOT

CREATE-Statement:

```
CREATE TABLE zeitslot (  
  zeitslot_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  datum      DATE NOT NULL,  
  startzeit  TIME NOT NULL,  
  endzeit    TIME NOT NULL,  
  CHECK (startzeit < endzeit)  
);
```

Testfall: Freie Zeitslots am 2026-10-02 →
10:00–11:00 und 16:00–17:00.

Query Results

SQL Query Editor (click to collapse)

SELECT zeitslot_id, startzeit, endzeit FROM zeitslot WHERE datum = '2026-10-02';

Execute Query Clear

Filter results: 2 rows returned

zeitslot_id	startzeit	endzeit
1	10:00	11:00
2	16:00	17:00

Tabelle: STANDORT

CREATE-Statement:

```
CREATE TABLE standort (  
  standort_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  bezeichnung VARCHAR(100) NOT NULL,  
  strasse VARCHAR(100),  
  plz VARCHAR(10),  
  stadt VARCHAR(50) NOT NULL,  
  Breitengrad DECIMAL(9,6) NOT NULL  
    CHECK (Breitengrad BETWEEN -90 AND 90),  
  laengengrad DECIMAL(9,6) NOT NULL  
    CHECK (laengengrad BETWEEN -180 AND 180)  
);
```

Testfall: Standorte in Berlin →
Stadtbibliothek Mitte, Breite Strasse
30.



Query Results

SQL Query Editor (click to expand)

SELECT bezeichnung, strasse, stadt FROM standort WHERE stadt = 'Berlin';

Execute Query | Close

Filter results

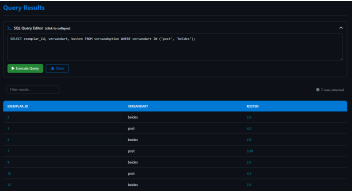
bezeichnung	strasse	stadt
Stadtbibliothek Mitte	Breite Strasse 30	Berlin

Tabelle: VERSANDOPTION

CREATE-Statement:

```
CREATE TABLE versandoption (  
  versandoption_id INTEGER  
    PRIMARY KEY AUTOINCREMENT,  
  exemplar_id INTEGER NOT NULL  
    UNIQUE REFERENCES  
      buchexemplar(exemplar_id)  
    ON DELETE CASCADE,  
  versandart VARCHAR(10) NOT NULL  
    CHECK (versandart IN  
      ('abholung', 'post', 'beides')),  
  kosten DECIMAL(6,2) NOT NULL  
    DEFAULT 0.00 CHECK (kosten >= 0)  
);
```

Testfall: Alle per Post versendbaren
Exemplare (versandart IN
('post', 'beides')) – 7 Treffer.



Query Results

SQL Query Editor (SQLite-Client)

SELECT exemplar_id, versandart, kosten FROM versandoption WHERE versandart IN ('post', 'beides');

Execute Query | Close

Filter results: | 7 rows returned

exemplar_id	versandart	kosten
1	beides	0.00
2	post	0.00
3	beides	0.00
4	post	0.00
5	beides	0.00
6	post	0.00
7	beides	0.00

Tabelle: BENACHRICHTIGUNG

CREATE-Statement:

```
CREATE TABLE benachrichtigung (  
  benachrichtigung_id INTEGER  
    PRIMARY KEY AUTOINCREMENT,  
  nutzer_id INTEGER NOT NULL  
    REFERENCES nutzer(nutzer_id)  
    ON DELETE CASCADE,  
  typ VARCHAR(30) NOT NULL,  
  inhalt TEXT NOT NULL,  
  gelesen BOOLEAN NOT NULL DEFAULT 0  
    CHECK (gelesen IN (0,1)),  
  erstellt_am DATETIME NOT NULL  
    DEFAULT CURRENT_TIMESTAMP  
);
```

Testfall: Ungelesene
Benachrichtigungen von Ivo Neumann
→ *Ein gewünschtes Buch ist jetzt
verfügbar.*

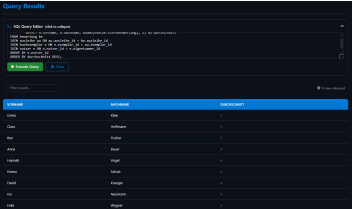


Tabelle: BEWERTUNG

CREATE-Statement:

```
CREATE TABLE bewertung (  
  bewertung_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  ausleihe_id INTEGER NOT NULL  
    UNIQUE REFERENCES  
      ausleihe(ausleihe_id)  
    ON DELETE CASCADE,  
  sternebewertung INTEGER NOT NULL  
    CHECK (sternebewertung  
      BETWEEN 1 AND 5),  
  kommentar TEXT,  
  bewertungsdatum DATE NOT NULL  
    DEFAULT CURRENT_DATE  
);
```

Testfall: Durchschnittsbewertung je Eigentüemer:in (GROUP BY) – Top: Greta Klein, Clara Hoffmann, Ben Fischer je 5.0. *ausleihe_id* ist UNIQUE → höchstens eine Bewertung je Ausleihe (Feedback Tutor Punkt 1).



Query Results

SQL Query Editor (SQLite)

```
SELECT owner_id, owner_name, ROUND(AVG(sternebewertung), 1) AS avg_rating  
FROM bewertung  
GROUP BY owner_id, owner_name  
ORDER BY avg_rating DESC
```

Run Query

Table Results

owner_id	owner_name	avg_rating
1	Greta Klein	5.0
2	Clara Hoffmann	5.0
3	Ben Fischer	5.0
4	Anna Müller	4.5
5	Max Schmidt	4.0
6	Lena Weber	3.5
7	David Wagner	3.0
8	Michaela Klein	2.5
9	Julia Meyer	2.0

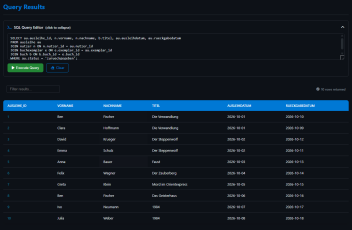
Dreifachbeziehungen

Dreifachbeziehung: AUSLEIHE (LEIHT_AUS)

CREATE-Statement:

```
CREATE TABLE ausleihe (  
  ausleihe_id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
  nutzer_id   INTEGER NOT NULL  
    REFERENCES nutzer(nutzer_id),  
  exemplar_id INTEGER NOT NULL  
    REFERENCES  
      buchexemplar(exemplar_id),  
  zeitslot_id INTEGER NOT NULL  
    REFERENCES zeitslot(zeitslot_id),  
  ausleihdatum DATE NOT NULL,  
  rueckgabedatum DATE,  
  status VARCHAR(15) NOT NULL  
    DEFAULT 'aktiv'  
    CHECK (status IN ('aktiv',  
      'zurueckgegeben', 'ueberfaellig')),  
  CHECK (rueckgabedatum IS NULL OR  
    rueckgabedatum >= ausleihdatum),  
  UNIQUE (exemplar_id, zeitslot_id)  
);
```

Testfall: Alle abgeschlossenen
Ausleihen mit Entleiher:in und
Buchtitel – 10 Treffer (Nutzer:in ×
Buchexemplar × Zeitslot).



Query Results

SQL Query Editor (click to edit)

```
SELECT ausleihe_id, nutzer_id, nutzer_name, zeitslot_id, zeitslot_name, buchexemplar_id, buchexemplar_title  
FROM ausleihe  
WHERE (ausleihe_id > 0) AND (nutzer_id > 0) AND (zeitslot_id > 0) AND (buchexemplar_id > 0)  
ORDER BY ausleihe_id
```

Execute Query Close

10 rows returned

ausleihe_id	nutzer_id	nutzer_name	zeitslot_id	zeitslot_name	buchexemplar_id	buchexemplar_title
1	Ben	Pfeifer	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
2	Clara	Hoffmann	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
3	David	Wagner	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
4	Emma	Schulz	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
5	Alex	Bauer	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
6	Julia	Wagner	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
7	Clara	Ben	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
8	Ben	Pfeifer	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
9	Ben	Pfeifer	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01
10	Julia	Wagner	2024-10-01	Ein Handbuch	2024-10-01	2024-10-01

Dreifachbeziehung: VERFUEGBARKEIT (VERFUEGBAR_AN)

CREATE-Statement:

```
CREATE TABLE verfuegbarkeit (  
  verfuegbarkeit_id INTEGER  
    PRIMARY KEY AUTOINCREMENT,  
  exemplar_id INTEGER NOT NULL  
    REFERENCES  
      buchexemplar(exemplar_id)  
    ON DELETE CASCADE,  
  zeitslot_id INTEGER NOT NULL  
    REFERENCES zeitslot(zeitslot_id)  
    ON DELETE CASCADE,  
  standort_id INTEGER NOT NULL  
    REFERENCES standort(standort_id),  
  verfuegbar BOOLEAN NOT NULL  
    DEFAULT 1  
    CHECK (verfuegbar IN (0,1)),  
  UNIQUE (exemplar_id, zeitslot_id,  
          standort_id)  
);
```

Testfall: Verfuegbare Exemplare an
der *Stadtbibliothek Mitte* –
kombiniert Buchexemplar, Zeitslot
und Standort zu einer
Verfuegbarkeitsaussage.



Query Results

SQL Query Editor: select * from verfuegbarkeit

Query Results: 3 rows, 3 columns

exemplar_id	date	count
1	2023-10-01	1000
2	2023-10-01	1000
3	2023-10-01	1000

M:N-Verknuepfungstabellen: BUCH_AUTOR und BUCH_GENRE

CREATE-Statements:

```
CREATE TABLE buch_autor (  
  buch_id    INTEGER NOT NULL REFERENCES buch(buch_id)    ON DELETE CASCADE,  
  autor_id   INTEGER NOT NULL REFERENCES autor(autor_id)  ON DELETE CASCADE,  
  PRIMARY KEY (buch_id, autor_id)  
);  
CREATE TABLE buch_genre (  
  buch_id    INTEGER NOT NULL REFERENCES buch(buch_id)    ON DELETE CASCADE,  
  genre_id   INTEGER NOT NULL REFERENCES genre(genre_id)  ON DELETE CASCADE,  
  PRIMARY KEY (buch_id, genre_id)  
);
```

Loesen die binaeren M:N-Beziehungen *Verfasst* und *Gehoert zu* aus dem ER-Modell (Phase 1) relational auf. Zusammengesetzter Primaerschluessel verhindert doppelte Zuordnungen.

Constraints und Geschäftsregeln

Trigger fuer Geschaeftsregeln (nicht per CHECK abbildbar)

SQLite erlaubt keine Subqueries in CHECK-Constraints – Regeln, die auf andere Tabellen verweisen, werden daher per TRIGGER durchgesetzt:

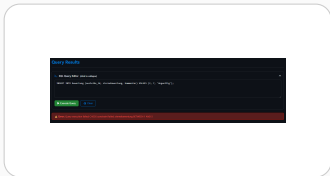
```
CREATE TRIGGER trg_ausleihe_kein_eigenausleih
BEFORE INSERT ON ausleihe FOR EACH ROW
WHEN NEW.nutzer_id = (SELECT eigentuemer_id FROM buchexemplar
                      WHERE exemplar_id = NEW.exemplar_id)
BEGIN SELECT RAISE(ABORT, 'Nutzer_kann_eigenes_Buchexemplar_nicht_ausleihen'); END;

CREATE TRIGGER trg_bewertung_nur_nach_rueckgabe
BEFORE INSERT ON bewertung FOR EACH ROW
WHEN (SELECT status FROM ausleihe WHERE ausleihe_id = NEW.ausleihe_id) != 'zurueckgegeben'
BEGIN SELECT RAISE(ABORT, 'Bewertung_nur_fuer_abgeschlossene_Ausleihen_erlaubt'); END;
```

Negativtestfaelle: Geschaeftsregeln aktiv geprueft (1/2)

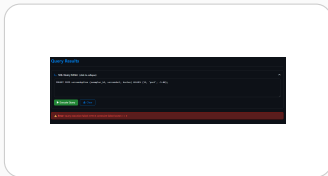
Laut Tutor-Feedback wurden gezielt *ungueltige* Daten getestet und tatsaechlich in SQLite ausgefuehrt – alle sechs wie erwartet abgelehnt:

(1) Sternebewertung = 7



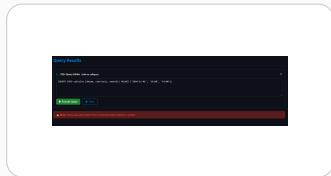
CHECK: sternebewertung BETWEEN 1 AND 5

(2) Versandkosten = -5.00



CHECK: kosten $\geq$ 0

(3) Zeitslot 15:00–14:00



CHECK: startzeit < endzeit

Negativtestfaelle: Geschaeftsregeln aktiv geprueft (2/2)

(4) Ausleihe mit nutzer_id = 9999



FOREIGN KEY constraint failed (mit PRAGMA
foreign_keys = ON;)

(5) Eigentuemer:in leiht eigenes Exemplar



Trigger: Nutzer kann eigenes Buchexemplar
nicht ausleihen

(6) Bewertung zu aktiver Ausleihe



Trigger: Bewertung nur fuer abgeschlossene
Ausleihen erlaubt

Hinweis: SQLite erzwingt Fremdschlüssel nur, wenn PRAGMA foreign_keys = ON; pro Verbindung gesetzt ist – ohne dieses PRAGMA wuerde Test (4) faelschlich durchgehen, obwohl das Schema korrekt ist.

Zusammenfassung und Reflexion

Zusammenfassung der Implementierung

Das ER-Modell aus Phase 1 (13 Entitäten, 2 Dreifachbeziehungen) wurde vollständig in SQLite als 17 physische Tabellen umgesetzt: 13 Entitätstabellen, 2 Dreifachbeziehungstabellen (ausleihe, verfuegbarkeit) und 2 M:N-Verknuepfungstabellen (buch_autor, buch_genre).

Alle 17 Tabellen enthalten mindestens 10 realistische Dummy-Datensätze (insgesamt > 190 Zeilen), erzeugt und verifiziert per Python-sqlite3-Skript. Datenintegrität wird durch NOT NULL, UNIQUE, Fremdschlüssel (inkl. ON DELETE CASCADE wo fachlich sinnvoll) sowie CHECK-Constraints fuer alle relevanten Wertebereiche sichergestellt; zwei Trigger decken Geschäftsregeln ab, die CHECK allein nicht abbilden kann.

Fuer jede Entität liegt mindestens ein erfolgreicher Testfall vor; zusätzlich wurden 6 gezielt ungültige Fälle formuliert und ausgeführt, die alle korrekt abgelehnt wurden – direkte Umsetzung des Tutor-Feedbacks aus Phase 1.

Herausforderungen und Erkenntnisse

Werk vs. Exemplar: Erst durch das Tutor-Feedback wurde klar, dass `eigentuemer_id` nicht auf BUCH gehoert, sondern auf ein eigenes BUCHEXEMPLAR – sonst koennte ein Buchtitel nur einer Person gehoeren.

Bewertung ohne Ausleihe verhindern: Eine reine Dreifachbeziehung Bewertender–Buch–Eigentuemer haette nicht ausgeschlossen, dass ohne echten Ausleihvorgang bewertet wird. Die Kopplung ueber `ausleihe_id` (UNIQUE) loest das strukturell, nicht nur per Anwendungslogik.

SQLite-Grenzen: Keine nativen ENUM-Typen und keine Subqueries in CHECK-Constraints – das erzwang saubere CHECK-Listen bzw. Trigger, was die Constraints expliziter macht als ein einfacher ENUM-Typ.

Naechster Schritt (Phase 3): Weitere Indizes fuer Abfrageperformance sowie Ausbau der Umkreissuche (Haversine-Berechnung auf Basis von `standort`).

Vielen Dank für die Aufmerksamkeit!

Fragen?